JWT Authentication

Implementation guide for the expense-tracker NestJS backend

This guide adds login with JSON Web Tokens to the backend. Every route requires a valid token except registration and login. The code follows the conventions in CLAUDE.md: Prisma only in repositories, services throw domain errors, error bodies are `{ code, message }`, config comes from `ConfigService`, and controllers never touch repositories.

What you will build

Endpoint	Access	Purpose
`POST /auth/register`	Public	Create an account (and its group) with a password. Returns a session.
`POST /auth/login`	Public	Exchange email + password for a session (token + user).
`POST /users`	Removed	Account creation moves to the auth module.
`GET /users/:id`	Token	Returns the user, including `groupId`.
`PATCH /users/:id`	Token	Update profile or move to another group.
`DELETE /users/:id`	Token	Delete the user.
`POST /expenses` (future)	Token	Uses the caller's `groupId` from the database, never from the body.

Key design decisions

- **The token carries only the user id** (`sub`). A user's group can change, so a `groupId` inside a token would go stale and keep granting access to the old group until it expired.
- **The guard loads the user on every request.** It is one indexed lookup, and it also rejects tokens of users that were deleted after the token was issued.
- **Secure by default.** A global guard protects every route; public routes opt out with `@Public()`.
- **Short-lived access tokens** (15 minutes by default). Refresh tokens are covered as an optional extension at the end.
- **argon2** hashes passwords. The hash never leaves the repository layer except as an explicit credentials type, and the mapper drops it from `User`.
- **No Passport.** `@nestjs/jwt` plus a small custom guard is less code and fewer abstractions for a single strategy. Flutter and Vue will do Google/Facebook sign-in with the providers' own SDKs, so the backend only verifies their tokens; no redirect flow, so Passport is not needed later either.
- **Open for social login.** Every login method ends the same way: the backend issues *its own* token. Google and Facebook will be extra endpoints next to `/auth/login`; the guard, token and `users` table do not change when they arrive. Section 12 shows what that step adds.

Three choices that keep the door open
1. `passwordHash` is nullable from day one: a future Google-only user has no password.
2. Registration lives in `POST /auth/register`, so every way of creating an account sits in one module.
3. The `UserIdentity` table for providers is **not** built now; it arrives with the first provider, as a pure addition.

Steps at a glance

#	Step	Main files
1	Install dependencies	`package.json`
2	Add JWT configuration	`config/*.ts`, `.env`
3	Nullable password hash	`prisma/schema.prisma` + migration
4	Password hashing service	`modules/auth/password-hasher.ts`
5	401 error base class	`shared/errors/`, `domain-exception.filter.ts`
6	Users module: accept a hash, drop POST /users	`modules/users/`
7	Auth module: register, login, token verification	`modules/auth/`
8	Protect routes and use the current user	`user.controller.ts`, expenses
9	Test end to end	curl
12	Later: Google and Facebook	`modules/auth/`, `UserIdentity`

1. Install dependencies

```
yarn add @nestjs/jwt argon2
```

`argon2` ships prebuilt binaries for macOS, Linux and Windows, so no compiler is normally needed. If your Docker image is Alpine-based and the install fails, add `python3 make g++` to the build stage.

2. Configuration

Two new environment variables. The secret must be long and random; generate one with `openssl rand -base64 48`. Never commit the real value.

.env and .env.example

```
JWT_SECRET="replace-with-output-of-openssl-rand-base64-48"
JWT_EXPIRES_IN_SECONDS=900
```

src/config/config.types.ts

```
export type AuthConfig = {
  readonly jwtSecret: string;
  readonly jwtExpiresInSeconds: number;
};

export type Config = {
  readonly app: AppConfig;
  readonly database: DatabaseConfig;
  readonly auth: AuthConfig;
};
```

Validation follows the existing `ParseResult` pattern. Add the two fields to `Env`, the parsers below, and include them in `validateEnv` the same way `appPort` and `databaseUrl` are handled today.

src/config/env.validation.ts (additions)

```
const MIN_JWT_SECRET_LENGTH = 32;
const DEFAULT_JWT_EXPIRES_IN_SECONDS = 900;

export function validateEnv(raw: Record<string, unknown>): Env {
  const appPort = parseAppPort(raw.APP_PORT);
  const databaseUrl = parseDatabaseUrl(raw.DATABASE_URL);
  const jwtSecret = parseJwtSecret(raw.JWT_SECRET);
  const jwtExpiresIn = parseJwtExpiresIn(raw.JWT_EXPIRES_IN_SECONDS);

  if (!appPort.ok || !databaseUrl.ok || !jwtSecret.ok || !jwtExpiresIn.ok) {
    const errors = [appPort, databaseUrl, jwtSecret, jwtExpiresIn]
      .filter((result): result is ParseFailure => !result.ok)
      .map((failure) => failure.error);

    throw new Error(
      `Invalid environment configuration:\n  - ${errors.join('\n  - ')}`,
    );
  }

  return {
    APP_PORT: appPort.value,
    DATABASE_URL: databaseUrl.value,
    JWT_SECRET: jwtSecret.value,
    JWT_EXPIRES_IN_SECONDS: jwtExpiresIn.value,
  };
}

function parseJwtSecret(raw: unknown): ParseResult<string> {
  if (typeof raw !== 'string' || raw.length < MIN_JWT_SECRET_LENGTH) {
    return {
      ok: false,
      error: `JWT_SECRET must be at least ${MIN_JWT_SECRET_LENGTH} characters`,
    };
  }
  return { ok: true, value: raw };
}

function parseJwtExpiresIn(raw: unknown): ParseResult<number> {
```

```
  if (raw === undefined || raw === '') {
    return { ok: true, value: DEFAULT_JWT_EXPIRES_IN_SECONDS };
  }

  const seconds = Number(raw);
  if (!Number.isInteger(seconds) || seconds <= 0) {
    return { ok: false, error: 'JWT_EXPIRES_IN_SECONDS must be a positive integer' };
  }
  return { ok: true, value: seconds };
}
```

src/config/configuration.ts (inside the returned object)

```
auth: {
  jwtSecret: env.JWT_SECRET,
  jwtExpiresInSeconds: env.JWT_EXPIRES_IN_SECONDS,
},
```

3. A nullable password hash

prisma/schema.prisma

```
model User {
  id           String   @id @default(uuid()) @db.Uuid
  firstName    String   @map("first_name")
  lastName     String?  @map("last_name")
  email        String   @unique
  passwordHash String?  @map("password_hash")
  // ...createdAt, updatedAt, groupMembership unchanged
}
```

```
yarn db:migrate --name add_user_password_hash
```

Nullable on purpose. Today every account registers with a password, but a user who later signs up with Google has none. Making it optional now avoids a migration on a table with real data later, and adding a nullable column never fails on existing rows.

Login treats a missing hash exactly like a wrong password, so an account without a password can never be entered with one, and the response does not reveal how the account was created.

4. Password hashing service

Only the auth module creates and checks credentials, so the hasher lives there. It wraps argon2 so the rest of the code never imports the library directly and tests can replace it.

src/modules/auth/password-hasher.ts

```
import { Injectable } from '@nestjs/common';
import { hash, verify } from 'argon2';

@Injectable()
export class PasswordHasher {
  hash(password: string): Promise<string> {
    return hash(password);
  }

  verify(passwordHash: string, password: string): Promise<boolean> {
    return verify(passwordHash, password);
  }
}
```

It is registered as a provider of `AuthModule` (section 7).

5. A base class for 401 errors

The error filter maps by base class. Authentication failures need a new base, `UnauthorizedError`, mapped to 401. This is a one-time filter edit for a new *kind* of error; individual codes still never touch the filter.

src/shared/errors/unauthorized.error.ts

```
import { DomainError } from './domain.error';

export abstract class UnauthorizedError extends DomainError {}
```

src/shared/filters/domain-exception.filter.ts

```
function toStatus(error: DomainError): HttpStatus {
  if (error instanceof UnauthorizedError) {
    return HttpStatus.UNAUTHORIZED;
  }
  if (error instanceof NotFoundError) {
    return HttpStatus.NOT_FOUND;
  }
  // ...ConflictError, ValidationError unchanged
}
```

6. Users module: accept a hash, drop POST /users

The users module keeps owning user and group creation, but it no longer receives HTTP requests to do it and never sees a plain password. The auth module hashes first and hands over a ready-made `NewUser`. A future Google signup will call the same method with `passwordHash: null`.

Input type for creating a user

src/modules/users/domain/new-user.ts

```
export type NewUser = {
  readonly firstName: string;
  readonly lastName: string | null;
  readonly email: string;
  readonly passwordHash: string | null;
  readonly groupId?: string;
};
```

Service

`create` takes `NewUser` instead of `CreateUserDto`; the group logic (new group, or join `groupId`) stays exactly as it is today.

src/modules/users/user.service.ts

```
async create(newUser: NewUser): Promise<User> {
  const { groupId, ...data } = newUser;
  const user =
    groupId === undefined
      ? await this.createWithNewGroup(data)
      : await this.createInGroup(data, groupId);
  // ...logging unchanged
}

// Returns null instead of throwing: login must not reveal whether the email exists,
// so the caller turns "no user", "no password" and "wrong password" into one error.
findPasswordCredentials(email: string): Promise<PasswordCredentials | null> {
  return this.userRepository.findPasswordCredentials(email);
}
```

Credentials type and repository

Login needs the hash, but `User` must never carry it. A separate, narrow type makes that explicit. Accounts without a password come back as `null`, indistinguishable from an unknown email:

src/modules/users/domain/password-credentials.ts

```
export type PasswordCredentials = {
  readonly userId: string;
  readonly passwordHash: string;
};
```

src/modules/users/user.repository.ts

```
export type CreateUserData = {
  readonly firstName: string;
  readonly lastName: string | null;
  readonly email: string;
  readonly passwordHash: string | null;
};

async findPasswordCredentials(email: string): Promise<PasswordCredentials | null> {
  const record = await this.prisma.user.findUnique({
    where: { email },
    select: { id: true, passwordHash: true },
  });
  if (record === null || record.passwordHash === null) {
    return null;
  }
  return { userId: record.id, passwordHash: record.passwordHash };
}
```

The mapper needs no change: `UserRecord` does not list `passwordHash`, so `toDomain` drops it. Responses can never leak it.

Controller and DTO

- Delete the `@Post()` handler from `user.controller.ts`.
- Delete `dto/create-user.dto.ts`; its fields move to `RegisterDto` in the auth module.
- Add the password limits next to the other field constants:

src/modules/users/dto/user-field.constants.ts (add)

```
export const USER_PASSWORD_MIN_LENGTH = 8;
export const USER_PASSWORD_MAX_LENGTH = 128;
```

The max length bounds hashing cost so nobody can send a 10 MB password. `UpdateUserDto` does **not** get a password field; changing a password deserves its own endpoint that asks for the current one.

7. The auth module

```
src/modules/auth/
  domain/
    errors.ts
    access-token-payload.ts
    access-token.ts
    auth-session.ts
  dto/
    register.dto.ts
    login.dto.ts
    auth-session-response.dto.ts
  decorators/
    public.decorator.ts
    current-user.decorator.ts
  guards/
    jwt-auth.guard.ts
  authenticated-request.ts
  password-hasher.ts
  auth.controller.ts
  auth.service.ts
  auth.mapper.ts
  auth.module.ts
```

Register and login both return a **session**: the access token plus the user, including `groupId`. Flutter and Vue get everything they need from one call, and the future Google/Facebook endpoints will return exactly the same shape.

Domain

src/modules/auth/domain/errors.ts

```ts
import { UnauthorizedError } from '../../../shared/errors/unauthorized.error';

export const AuthErrorCode = {
  INVALID_CREDENTIALS: 'AUTH_INVALID_CREDENTIALS',
  TOKEN_MISSING: 'AUTH_TOKEN_MISSING',
  TOKEN_INVALID: 'AUTH_TOKEN_INVALID',
} as const;

export type AuthErrorCode = (typeof AuthErrorCode)[keyof typeof AuthErrorCode];

export class InvalidCredentialsError extends UnauthorizedError {
  readonly code: AuthErrorCode = AuthErrorCode.INVALID_CREDENTIALS;

  constructor() {
    super('Email or password is incorrect');
  }
}

export class MissingTokenError extends UnauthorizedError {
  readonly code: AuthErrorCode = AuthErrorCode.TOKEN_MISSING;

  constructor() {
    super('Authorization header with a Bearer token is required');
  }
}

export class InvalidTokenError extends UnauthorizedError {
  readonly code: AuthErrorCode = AuthErrorCode.TOKEN_INVALID;

  constructor() {
    super('Access token is invalid or expired');
  }
}
```

src/modules/auth/domain/access-token-payload.ts

```
export type AccessTokenPayload = {
  readonly sub: string;
};
```

src/modules/auth/domain/access-token.ts

```
export type AccessToken = {
  readonly token: string;
  readonly expiresInSeconds: number;
};
```

src/modules/auth/domain/auth-session.ts

```
import type { User } from '../../users/domain/user';
import type { AccessToken } from './access-token';

export type AuthSession = {
  readonly user: User;
  readonly accessToken: AccessToken;
};
```

DTOs and mapper

src/modules/auth/dto/register.dto.ts

```
import {
  IsEmail,
  IsNotEmpty,
  IsOptional,
  IsString,
  IsUUID,
  MaxLength,
  MinLength,
} from 'class-validator';
import {
  USER_EMAIL_MAX_LENGTH,
  USER_NAME_MAX_LENGTH,
  USER_PASSWORD_MAX_LENGTH,
  USER_PASSWORD_MIN_LENGTH,
} from '../../users/dto/user-field.constants';

export class RegisterDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(USER_NAME_MAX_LENGTH)
  readonly firstName!: string;

  @IsOptional()
  @IsString()
  @MaxLength(USER_NAME_MAX_LENGTH)
  readonly lastName?: string | null;

  @IsEmail()
  @MaxLength(USER_EMAIL_MAX_LENGTH)
  readonly email!: string;

  @IsString()
  @MinLength(USER_PASSWORD_MIN_LENGTH)
  @MaxLength(USER_PASSWORD_MAX_LENGTH)
  readonly password!: string;

  @IsOptional()
  @IsUUID()
```

```
  readonly groupId?: string;
}
```

These are the fields of today's `CreateUserDto` plus `password`. Joining a group with `groupId` keeps working exactly as it does now.

src/modules/auth/dto/login.dto.ts

```
import { IsEmail, IsString, MaxLength } from 'class-validator';
import {
  USER_EMAIL_MAX_LENGTH,
  USER_PASSWORD_MAX_LENGTH,
} from '../../users/dto/user-field.constants';

export class LoginDto {
  @IsEmail()
  @MaxLength(USER_EMAIL_MAX_LENGTH)
  readonly email!: string;

  @IsString()
  @MaxLength(USER_PASSWORD_MAX_LENGTH)
  readonly password!: string;
}
```

Login does not enforce the minimum length: that rule belongs to registration, and rejecting a short password at login with a validation error would tell an attacker something the generic credentials error does not.

src/modules/auth/dto/auth-session-response.dto.ts

```
import type { UserResponseDto } from '../../users/dto/user-response.dto';

export type AuthSessionResponseDto = {
  readonly accessToken: string;
  readonly tokenType: 'Bearer';
  readonly expiresIn: number;
  readonly user: UserResponseDto;
};
```

src/modules/auth/auth.mapper.ts

```
import { userMapper } from '../users/user.mapper';
import type { AuthSession } from './domain/auth-session';
import type { AuthSessionResponseDto } from './dto/auth-session-response.dto';

export const authMapper = {
  toResponse(session: AuthSession): AuthSessionResponseDto {
    return {
      accessToken: session.accessToken.token,
      tokenType: 'Bearer',
      expiresIn: session.accessToken.expiresInSeconds,
      user: userMapper.toResponse(session.user),
    };
  },
};
```

Service

Three public operations: `register` and `login` end in a session, `authenticate` turns a token back into a `User` for the guard. Every way of logging in funnels through the private `startSession`; a future `loginWithGoogle` is just one more caller. Login and token failures are always a generic 401 so nothing about accounts leaks.

src/modules/auth/auth.service.ts

```
import { Injectable, Logger } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { JwtService } from '@nestjs/jwt';
import type { Config } from '../../config/config.types';
import { UserNotFoundError } from '../users/domain/errors';
import type { User } from '../users/domain/user';
import { UserService } from '../users/user.service';
import type { AccessTokenPayload } from './domain/access-token-payload';
import type { AuthSession } from './domain/auth-session';
import { InvalidCredentialsError, InvalidTokenError } from './domain/errors';
import type { LoginDto } from './dto/login.dto';
import type { RegisterDto } from './dto/register.dto';
import { PasswordHasher } from './password-hasher';

@Injectable()
export class AuthService {
  private readonly logger = new Logger(AuthService.name);

  constructor(
    private readonly userService: UserService,
    private readonly passwordHasher: PasswordHasher,
    private readonly jwtService: JwtService,
    private readonly configService: ConfigService<Config, true>,
  ) {}

  async register(dto: RegisterDto): Promise<AuthSession> {
    const user = await this.userService.create({
      firstName: dto.firstName,
      lastName: dto.lastName ?? null,
      email: dto.email,
      passwordHash: await this.passwordHasher.hash(dto.password),
      groupId: dto.groupId,
    });

    this.logger.log({ msg: 'User registered', userId: user.id, method: 'password' });
    return this.startSession(user);
  }
```

```
  async login(dto: LoginDto): Promise<AuthSession> {
    const userId = await this.verifyCredentials(dto.email, dto.password);
    const user = await this.userService.findById(userId);

    this.logger.log({ msg: 'User logged in', userId, method: 'password' });
    return this.startSession(user);
  }

  async authenticate(token: string): Promise<User> {
    const payload = await this.verifyToken(token);
    return this.findTokenOwner(payload.sub);
  }

  private async startSession(user: User): Promise<AuthSession> {
    const payload: AccessTokenPayload = { sub: user.id };
    const token = await this.jwtService.signAsync(payload);
    return {
      user,
      accessToken: {
        token,
        expiresInSeconds: this.configService.get('auth.jwtExpiresInSeconds', {
          infer: true,
        }),
      },
    };
  }

  private async verifyCredentials(email: string, password: string): Promise<string> {
    const credentials = await this.userService.findPasswordCredentials(email);
    if (credentials === null) {
      throw new InvalidCredentialsError();
    }

    const isPasswordValid = await this.passwordHasher.verify(
      credentials.passwordHash,
      password,
    );
    if (!isPasswordValid) {
```

```
      throw new InvalidCredentialsError();
    }
    return credentials.userId;
  }

  private async verifyToken(token: string): Promise<AccessTokenPayload> {
    try {
      return await this.jwtService.verifyAsync<AccessTokenPayload>(token);
    } catch {
      // Expired, malformed and wrongly signed tokens all get the same answer.
      throw new InvalidTokenError();
    }
  }

  private async findTokenOwner(userId: string): Promise<User> {
    try {
      return await this.userService.findById(userId);
    } catch (error) {
      if (error instanceof UserNotFoundError) {
        throw new InvalidTokenError();
      }
      throw error;
    }
  }
}
```

Request type and decorators

src/modules/auth/authenticated-request.ts

```
import type { Request } from 'express';
import type { User } from '../users/domain/user';

export type AuthenticatedRequest = Request & {
  user?: User;
};
```

src/modules/auth/decorators/public.decorator.ts

```
import { type CustomDecorator, SetMetadata } from '@nestjs/common';

export const IS_PUBLIC_KEY = 'isPublic';

export const Public = (): CustomDecorator => SetMetadata(IS_PUBLIC_KEY, true);
```

src/modules/auth/decorators/current-user.decorator.ts

```
import { createParamDecorator, type ExecutionContext } from '@nestjs/common';
import type { User } from '../../users/domain/user';
import type { AuthenticatedRequest } from '../authenticated-request';

export const CurrentUser = createParamDecorator(
  (_data: unknown, context: ExecutionContext): User => {
    const request = context.switchToHttp().getRequest<AuthenticatedRequest>();
    if (request.user === undefined) {
      throw new Error('@CurrentUser() used on a route that is not authenticated');
    }
    return request.user;
  },
);
```

The plain `Error` is deliberate: it can only happen if a developer puts `@CurrentUser()` on a `@Public()` route, so it should surface as a 500, not as a client error.

Guard

src/modules/auth/guards/jwt-auth.guard.ts

```
import { type CanActivate, type ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { AuthService } from '../auth.service';
import type { AuthenticatedRequest } from '../authenticated-request';
import { IS_PUBLIC_KEY } from '../decorators/public.decorator';
import { MissingTokenError } from '../domain/errors';

const BEARER_SCHEME = 'Bearer';

@Injectable()
export class JwtAuthGuard implements CanActivate {
  constructor(
    private readonly reflector: Reflector,
    private readonly authService: AuthService,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    if (this.isPublic(context)) {
      return true;
    }

    const request = context.switchToHttp().getRequest<AuthenticatedRequest>();
    const token = extractBearerToken(request.headers.authorization);
    request.user = await this.authService.authenticate(token);
    return true;
  }

  private isPublic(context: ExecutionContext): boolean {
    const isPublic = this.reflector.getAllAndOverride<boolean | undefined>(
      IS_PUBLIC_KEY,
      [context.getHandler(), context.getClass()],
    );
    return isPublic === true;
  }
}

function extractBearerToken(header: string | undefined): string {
  const [scheme, token] = header?.split(' ') ?? [];
```

```
  if (scheme !== BEARER_SCHEME || token === undefined || token === '') {
    throw new MissingTokenError();
  }
  return token;
}
```

Errors thrown in a guard go through the global filters like any other, so a missing or bad token becomes a 401 `{ code, message }` body automatically.

Controller and module

src/modules/auth/auth.controller.ts

```
import { Body, Controller, HttpCode, HttpStatus, Post } from '@nestjs/common';
import { authMapper } from './auth.mapper';
import { AuthService } from './auth.service';
import { Public } from './decorators/public.decorator';
import type { AuthSessionResponseDto } from './dto/auth-session-response.dto';
import { LoginDto } from './dto/login.dto';
import { RegisterDto } from './dto/register.dto';

@Public()
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('register')
  @HttpCode(HttpStatus.CREATED)
  async register(@Body() dto: RegisterDto): Promise<AuthSessionResponseDto> {
    const session = await this.authService.register(dto);
    return authMapper.toResponse(session);
  }

  @Post('login')
  @HttpCode(HttpStatus.OK)
  async login(@Body() dto: LoginDto): Promise<AuthSessionResponseDto> {
    const session = await this.authService.login(dto);
    return authMapper.toResponse(session);
  }
}
```

src/modules/auth/auth.module.ts

```
import { Module } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { APP_GUARD } from '@nestjs/core';
import { JwtModule } from '@nestjs/jwt';
import type { Config } from '../../config/config.types';
import { UserModule } from '../users/user.module';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { JwtAuthGuard } from './guards/jwt-auth.guard';
import { PasswordHasher } from './password-hasher';

const JWT_ALGORITHM = 'HS256';

@Module({
  imports: [
    UserModule,
    JwtModule.registerAsync({
      inject: [ConfigService],
      useFactory: (configService: ConfigService<Config, true>) => ({
        secret: configService.get('auth.jwtSecret', { infer: true }),
        signOptions: {
          algorithm: JWT_ALGORITHM,
          expiresIn: configService.get('auth.jwtExpiresInSeconds', { infer: true }),
        },
        verifyOptions: { algorithms: [JWT_ALGORITHM] },
      }),
    }),
  ],
  controllers: [AuthController],
  providers: [
    AuthService,
    PasswordHasher,
    { provide: APP_GUARD, useClass: JwtAuthGuard },
  ],
})
export class AuthModule {}
```

Register `AuthModule` in `AppModule`'s imports. Because the guard is provided as `APP_GUARD`, it applies to every controller in the app, including ones added later.

Why pin the algorithm? `verifyOptions.algorithms` stops a token signed with a different algorithm (including the infamous `none`) from ever being accepted.

8. Protect routes and use the current user

Users

Nothing to mark: with `POST /users` gone, every route in `UserController` needs a token, and the only public routes in the app are `/auth/register` and `/auth/login`.

A valid token is not the same as permission. With only the guard, any logged-in user can read, edit or delete *any* user. Decide the rule and enforce it in the service, for example:

- `GET/PATCH/DELETE /users/:id`: only allowed when `:id` equals the current user's id. Add a `ForbiddenError` base (mapped to 403) and a `UserAccessDeniedError` in the users module, or simpler: replace `:id` routes with `/users/me`.
- `GET /users`: listing every user is rarely right once there is auth. Restrict it to the caller's group or remove it.

Expenses (the reason for all this)

The expense endpoints take the group from the authenticated user, never from the request. The DTO has no `groupId` field at all, and `forbidNonWhitelisted` rejects one if a client sends it.

src/modules/expenses/expense.controller.ts (sketch)

```
@Controller('expenses')
export class ExpenseController {
  constructor(private readonly expenseService: ExpenseService) {}

  @Post()
  @HttpCode(HttpStatus.CREATED)
  async create(
    @CurrentUser() user: User,
    @Body() dto: CreateExpenseDto,
  ): Promise<ExpenseResponseDto> {
    const expense = await this.expenseService.create(user.groupId, dto);
    return expenseMapper.toResponse(expense);
  }
}
```

9. Test it end to end

```
# 1. Register (public)
curl -s -X POST localhost:3000/auth/register -H 'Content-Type: application/json' \
  -d '{"firstName":"Ana","email":"ana@example.com","password":"correct-horse-42"}'

# 2. Log in and keep the token and the user id
SESSION=$(curl -s -X POST localhost:3000/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"ana@example.com","password":"correct-horse-42"}')
TOKEN=$(echo "$SESSION" | jq -r .accessToken)
USER_ID=$(echo "$SESSION" | jq -r .user.id)

# 3. Call a protected route
curl -s localhost:3000/users/$USER_ID -H "Authorization: Bearer $TOKEN"

# 4. Without a token -> 401 AUTH_TOKEN_MISSING
curl -s -i localhost:3000/users/$USER_ID

# 5. Wrong password -> 401 AUTH_INVALID_CREDENTIALS
curl -s -i -X POST localhost:3000/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"ana@example.com","password":"wrong-password"}'

# 6. The old route is gone -> 404 NOT_FOUND (no handler matches, so the guard never runs)
curl -s -i -X POST localhost:3000/users -H 'Content-Type: application/json' -d '{}'
```

Expected responses

```
POST /auth/register  201   and   POST /auth/login  200
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "6f1c2a9e-0d4b-4c8e-9a51-3b7e2f0c1d22",
    "groupId": "b2e4d7a1-5c3f-4e8b-8f61-9d0a7c4e2b10",
    "firstName": "Ana",
    "lastName": null,
    "email": "ana@example.com",
    "createdAt": "2026-10-10T15:20:11.000Z",
    "updatedAt": "2026-10-10T15:20:11.000Z"
  }
}

POST /auth/register with an email already in use  409
{ "code": "USER_EMAIL_TAKEN", "message": "Email ana@example.com is already taken" }

GET /users/:id without header  401
{ "code": "AUTH_TOKEN_MISSING", "message": "Authorization header with a Bearer token is required" }

GET /users/:id with expired or tampered token  401
{ "code": "AUTH_TOKEN_INVALID", "message": "Access token is invalid or expired" }

POST /auth/login with unknown email or wrong password  401
{ "code": "AUTH_INVALID_CREDENTIALS", "message": "Email or password is incorrect" }
```

Also check: register without `password` gives 400 with a `fields.password` entry; the user response never contains a password field; deleting a user makes their still-unexpired token return `AUTH_TOKEN_INVALID`.

Unit tests worth writing

- `AuthService.login`: same error for unknown email, an account without a password, and a wrong password.
- `AuthService.register`: stores a hash, never the plain password; returns a session for the new user.
- `AuthService.authenticate`: deleted user gives `InvalidTokenError`, other errors are rethrown.
- `JwtAuthGuard`: public route skips verification; missing, malformed (`Basic abc`, `Bearer`) headers throw.
- `validateEnv`: short `JWT_SECRET` and non-integer expiry are reported together with other errors.

10. Error codes added

Code	Status	When
`AUTH_INVALID_CREDENTIALS`	401	Login with an unknown email or a wrong password.
`AUTH_TOKEN_MISSING`	401	No `Authorization` header, or not in `Bearer <token>` form.
`AUTH_TOKEN_INVALID`	401	Bad signature, expired, malformed, or the user no longer exists.

11. Security checklist

- Secret is at least 32 random characters, loaded from config, different per environment, never logged.
- Algorithm pinned to HS256 on both sign and verify.
- Access tokens expire in minutes, not days.

- Tokens and passwords never appear in logs. Log `userId`, not email.
- Login answers the same way for unknown email and wrong password.
- Rate-limit `POST /auth/login` (`@nestjs/throttler`) to slow down password guessing.
- HTTPS only in every non-local environment; tokens in plain HTTP can be stolen.
- Vue keeps the access token in memory, not `localStorage`. Flutter stores it with `flutter_secure_storage` (Keychain / Keystore), not `shared_preferences`.
- Authorization rules (who may act on which user or group) live in services, not just behind the guard.
- Optional hardening: when the email is unknown, still run `argon2.verify` against a dummy hash so response time does not reveal which emails are registered.

12. Later: Google and Facebook

Not part of this step. This section shows that nothing built above needs to change when they arrive.

How it will work

- **Client side:** Flutter uses `google_sign_in` / `flutter_facebook_auth`; Vue uses Google Identity Services and the Facebook JS SDK. Each gives the app a provider token.
- **Backend:** `POST /auth/google` receives the Google ID token and verifies it with `google-auth-library` (`verifyIdToken`: signature, audience = your client ids, expiry). `POST /auth/facebook` checks the Facebook access token against the Graph API.
- **Result:** both find or create the user, then call the same `startSession`, so the response is the same session shape as password login. The guard and every protected route are untouched.

What gets added then

prisma/schema.prisma (future)

```
model UserIdentity {
  provider       String   // 'GOOGLE' | 'FACEBOOK'
  providerUserId String   @map("provider_user_id")
  userId         String   @map("user_id") @db.Uuid
  createdAt      DateTime @default(now()) @map("created_at") @db.Timestamptz(6)

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@id([provider, providerUserId])
  @@index([userId])
  @@map("user_identities")
}
```

- **Match users by `(provider, providerUserId)`, never by email alone.** Emails change, and Facebook does not always return one.
- **First social login:** `UserService.create` with `passwordHash: null` (a new group is created as usual), then insert the identity, in one transaction.
- **Linking to an existing password account** by email only when the provider says the email is verified (Google's `email_verified`). Otherwise anyone could create a social profile with someone else's email and take over the account.
- **New error codes** such as `AUTH_PROVIDER_TOKEN_INVALID` in the same `AuthErrorCode` object.

13. Optional: refresh tokens

With 15-minute access tokens, users would log in constantly. The usual fix is a long-lived refresh token:

- Add a `RefreshToken` table: `id`, `userId`, `tokenHash`, `expiresAt`, `revokedAt`. Store only a hash of the token, like a password.
- Every session (password or social) returns both tokens. Vue: the refresh token goes in an HttpOnly, Secure, SameSite cookie. Flutter: in `flutter_secure_storage`.
- `POST /auth/refresh` (public) checks the stored hash, revokes the old token, issues a new pair (rotation). Reuse of a revoked token means theft: revoke all of that user's tokens.
- `POST /auth/logout` revokes the current refresh token. This is also how you get real logout, which pure stateless JWT cannot do.

Build this only once the basic flow works; everything in sections 1 to 9 stays the same, and so does the social login from section 12.